

Lect(11-13) JS Classes

JavaScript Objects

In JavaScript, an Object is a collection of key–value pairs (also called properties). Each property has a name (key) and a value that can be any data type — number, string, array, function, or even another object. Objects are used to store and organize data in a structured way.

Why Use Objects:

- To group related data together.
- To represent real-world entities (e.g., a car, user, student).
- To store dynamic data using key-value pairs.
- To create reusable data structures.

Ways to Create Objects in JavaScript

1. Object Literal Method (Most Common Way):

```
const student = {  
  name: "John",  
  age: 21,  
  course: "B.Tech",  
  isActive: true,  
  greet: function() {  
    return `Hello, my name is ${this.name}`;  
  }};  
delete student.course;  
console.log(student.name);  
console.log(student.greet());
```

Features:

- Simple and readable.
- Ideal for creating single objects.
- Properties and methods are defined directly.

2. Using new Object() Constructor :

```
const car = new Object();
car.brand = "Tesla";
car.model = "Model X";
car.price = 90000;
console.log(car.brand);
```

Features:

- Creates an empty object.
- Properties are added dynamically.
- Not often used in modern JS (literal syntax is preferred).

3. Using a Constructor Function

Used to create **multiple objects** of the same type.

```
function Person(name, age) {
  this.name = name;
  this.age = age;
  this.greet = function() {
    return `Hi, I'm ${this.name}`;
  };
}
const person1 = new Person("John", 21);
const person2 = new Person("Riya", 22);
console.log(person1.greet());
```

Features:

- Use `this` to refer to the current object.
- Use `new` keyword to create instances.
- Common before ES6 Classes.

4. Using ES6 class Syntax

A modern way to define object blueprints.

```

class User {
  constructor(name, email) {
    this.name = name;
    this.email = email;
  }
  display() {
    return `User: ${this.name}, Email: ${this.email}`;
  }
}
const user1 = new User("John", "xyz@example.com");
console.log(user1.display());

```

Features:

- Cleaner and more structured.
- Supports inheritance and encapsulation.
- Internally works like constructor functions.

5. Using Object.create()

Used to create a new object that **inherits** from another object.

```

const parent = {
  greet() {
    console.log("Hello from parent!");
  }
};
const child = Object.create(parent);
child.name = "Child Object";
child.greet();

```

Features:

- Great for prototypal inheritance.
- Sets up a prototype chain manually.

6. Using JSON (Data Exchange Format)

You can also define objects using **JSON (JavaScript Object Notation)** — a lightweight data format.

```
const jsonString = '{"name":"John","age":21,"course":"B.Tech"}';
const studentObj = JSON.parse(jsonString);
console.log(studentObj.name);
const backToJSON = JSON.stringify(studentObj); console.log(backToJSON);
```

Features:

- Commonly used for APIs and data storage.
- JSON is always in double quotes.

Nested Objects and Destructuring

Objects can be nested inside other objects. Destructuring helps extract nested properties easily.

```
let user = {
  id: 101,
  profile: {
    username: "john",
    contact: { email: "xyz@example.com" }
  }
};
let { profile: { username, contact: { email } } } = user;
console.log(username, email);
```

Object Cloning and Methods

To clone or merge objects, use spread operator or `Object.assign()`

Useful methods include:

- `Object.keys(obj)`
- `Object.values(obj)`
- `Object.entries(obj)`

```
let obj1 = {a:1, b:2};
let obj2 = {...obj1, c:3};
console.log(Object.keys(obj2));
console.log(Object.values(obj2));
console.log(Object.entries(obj2));
```

Object Table:

Concept	Description	Example
Object Literal	Simple object creation	<code>{name:"John"}</code>
Constructor	Function to create objects	<code>new Person("John")</code>
Class	ES6 syntax	<code>class User{}</code>
Object.create	Prototype-based inheritance	<code>Object.create(proto)</code>
JSON	Data format	<code>JSON.parse() / JSON.stringify()</code>
Methods	Keys holding functions	<code>obj.greet()</code>
Access	Dot or Bracket	<code>obj.key / obj["key"]</code>
Destructuring	Extract data	<code>const {name} = obj</code>

Forms

HTML forms are used to collect user input — text, email, password, checkbox, etc.

JavaScript helps us **control**, **validate**, and **process** this form data before it's submitted to the server.

```
<form id="loginForm">
  <input type="text" id="username" placeholder="Username">
  <input type="password" id="password" placeholder="Password">
  <button type="submit">Login</button>
</form>
<script>
const form = document.getElementById("loginForm");
form.addEventListener("submit", function(e) {
  e.preventDefault();
  const user = document.getElementById("username").value;
  const pass = document.getElementById("password").value;
  if (user === "" || pass === "") {alert("All fields are required!");}
  else if (pass.length < 6) {
    alert("Password must be at least 6 characters long!");
  }
  else {alert("Login successful!");
    form.reset();
  }
});</script>
```

Regular Expression (RegExp):

A Regular Expression (RegExp) is a pattern used to match strings or parts of strings. You can use it to search, validate, replace, or extract text in JavaScript.

Example:

```
let text = "Hello World";  
let pattern = /Hello/;  
console.log(pattern.test(text));
```

How to Create a RegExp in JavaScript :

There are two ways to create a Regular Expression:

1. Using Literal Syntax:

In JavaScript, you can create a Regular Expression directly by writing the pattern between two forward slashes `/.../` this is called the literal syntax.

It's the most common and simplest way to define a regular expression.

Syntax:

```
let regex = /pattern/;
```

Example:

```
let regex = /hello/;
```

2. Using RegExp Constructor:

The RegExp constructor allows you to create regular expressions dynamically using the `RegExp()` function.

Syntax:

```
let regex = new RegExp("hello");
```

Example:

```
let text = "HELLO hello";
let regex = /hello/gi;
console.log(text.match(regex));
```

Common Regex Components

Symbol	Meaning	Example	Matches
.	Any character except newline	/h.llo/	hello, hallo
^	Start of string	/^Hello/	Hello world
\$	End of string	/world\$/	Hello world
\d	Any digit (0–9)	/\d+/	123, 42
\w	Word character (a–z, A–Z, 0–9, _)	/\w+/	hello123
\s	Whitespace	/\s/	" "
[abc]	Match any character in brackets	/[aeiou]/	a, e, i
[^abc]	Match any character <i>not</i> in brackets	/[^aeiou]/	b, c, etc.
(abc)	Group	/(abc){2}/	abcabc
+	One or more	/\d+/	123
*	Zero or more	/\d*/	"", 123
?	Zero or one	/colou?r/	color, colour
{n,m}	Between n and m times	/\d{2,4}/	99, 2023

Examples:

1. Validate Email

```
let email = "test@example.com";  
let pattern = /^[\\w.-]+@[a-zA-Z_-]+?\\.([a-zA-Z]{2,3})$/;  
console.log(pattern.test(email));
```

2. Validate Phone Number

```
let phone = "9876543210";  
let pattern = /^[6-9]\\d{9}$/;  
console.log(pattern.test(phone));
```

3. Extract All Numbers

```
let text = "Price: 200, Discount: 50, Total: 150";  
let numbers = text.match(/\\d+/g);  
console.log(numbers);
```

4. Replace Multiple Spaces with One

```
let text = "Hello    world  JS";  
let result = text.replace(/\\s+/g, " ");  
console.log(result);
```

5. Find Words Starting with Capital Letter

```
let text = "JavaScript is Great";  
let pattern = /\\b[A-Z][a-z]*\\b/g;  
console.log(text.match(pattern));
```

Scenario-Based Problem: Student Registration Form

```
<!DOCTYPE html>
<html>
<head> <title>Student Registration</title> </head>
<body>
<form id="regForm">
  <input id="name" placeholder="Name"><br>
  <input id="email" placeholder="Email"><br>
  <input id="age" type="number" placeholder="Age"><br>
  <button type="submit">Register</button>
</form>
<script>
class Student {
  constructor(name, email, age) {
    this.name = name;
    this.email = email;
    this.age = age;
  }
}
let students = [];
document.getElementById("regForm").onsubmit = function(e) {
  e.preventDefault();
  let name = document.getElementById("name").value;
  let email = document.getElementById("email").value;
  let age = document.getElementById("age").value;
  let emailPattern = /^[^ ]+@[^ ]+\.[a-z]{2,3}$/;
  if (!emailPattern.test(email)) {
    alert("Invalid Email!");
    return;
  }
  let newStudent = new Student(name, email, age);
  students.push(newStudent);
  console.log("Registered Students:");
  for (let s of students) {
    console.log(`${s.name} - ${s.email} - ${s.age}`);
  }
};
</script>
</body>
</html>
```